

CSCI-UA 102 — Quiz 4

Trees

Name: _____ NetID: _____

The **height** of a node p in a tree is the number of edges on the longest downward path from p to a leaf (external node). A leaf has height 0. **Write an efficient, recursive method:**

```
static <E> int sumHeight(Tree<E> tree)
```

that calculates the sum of the heights of every node in the tree. *Hint: use an auxiliary variable.* You may use the following interfaces and assume working implementations exist.

```
public interface Position<E> {
    E getElement();
}
public interface Tree<E> {
    Position<E> root();
    Iterable<Position<E>> children(Position<E> p);
    boolean isExternal(Position<E> p);
    boolean isEmpty();
}
```

Solution

```
static <E> int sumHeight(Tree<E> tree) {
    if (tree.isEmpty()) return 0;
    return subtreeHeightAndSum(tree, tree.root())[1];
}

// Returns {height, sumOfHeights} for the subtree at p.
static <E> int[] subtreeHeightAndSum(Tree<E> tree,
                                     Position<E> p) {

    int height = 0;
    int sum = 0;
    for (Position<E> child : tree.children(p)) {
        int[] childResult = subtreeHeightAndSum(tree, child);
        height = Math.max(height, 1 + childResult[0]);
        sum += childResult[1];
    }
    sum += height;           // add this node's height
    return new int[]{height, sum};
}
```

Time complexity: $O(n)$ — each node is visited exactly once, with $O(c_p)$ work per node.

Why return two values? We need the height (to recurse) *and* the sum of heights (the answer). Returning {height, sumOfHeights} lets us compute both in a single bottom-up pass. Without this, computing the height for each node independently would be $O(n)$ per node, giving $O(n^2)$ total.